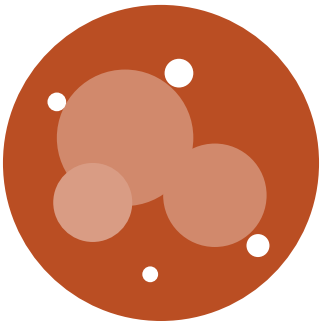
Marswalk

Planificación del algoritmo de ruta híbrido clásico + cuántico

NASA Space Apps Challenge · Reto Marswalk



Autor: Ángel Gabriel

Equipo: Cabezones

Rol: Algoritmo de ruta (híbrido clásico-cuántico)

Zona candidata: Cráter Jezero, Marte

Fecha: 21 de octubre de 2026

Índice

1. Resumen ejecutivo **1**

2. Arquitectura del sistema **1**

3. Equipo y contratos de intercambio **3**

4. Roadmap semanal de preparación **3**
4.1. Semana 1 — 21 al 27 de octubre: datos marcianos y mapa de pendientes 3
4.2. Semana 2 — 28 de octubre al 3 de noviembre: A* desde cero y validación 5
4.3. Semana 3 — 4 al 10 de noviembre: función de costo y restricciones duras 5
4.4. Semana 4 — 11 al 17 de noviembre: orienteering con OR-Tools y fundamentos cuánticos 6
4.5. Semana 5 — 18 al 24 de noviembre: QUBO, simulated annealing y calibración . . 7
4.6. Semana 6 — 25 de noviembre al 1 de diciembre: API, comparativa y QAOA opcional 8

5. Tabla resumen de entregables **10**

6. Plan hora por hora de las 48 horas **10**

7. Formulación matemática del QUBO del orienteering **11**

8. Limitaciones y honestidad científica **12**

9. Preguntas frecuentes de los jueces **13**

10. Checklist final de entrega **13**

A. Apéndice: estructura del repositorio e instalación **14**

1 Resumen ejecutivo

Este documento describe el plan técnico y de aprendizaje para el módulo de **algoritmo de ruta** del reto *Marswalk* del NASA Space Apps Challenge. El objetivo es entregar una herramienta que, a partir de datos de varias misiones (MOLA, HiRISE, CRISM, THEMIS, SHARAD, MEDA/REMS, RAD), proponga al astronauta una caminata segura y con alto valor científico cerca del cráter Jezero.

Objetivo

Construir un planificador de rutas con tres capas acopladas:

1. **Capa clásica:** construcción de un mapa de costos del terreno (pendiente, rugosidad, transitabilidad, radiación) a partir de un Modelo Digital de Elevación (DEM) y un camino mínimo con A* o Dijkstra.
2. **Capa de optimización combinatoria:** selección y ordenación de sitios científicos de interés mediante el *Orienteering Problem* (maximizar valor científico bajo un presupuesto de oxígeno y tiempo). Se utiliza Google OR-Tools como baseline clásico.
3. **Capa cuántica exploratoria:** formulación QUBO del mismo problema, resuelta con *simulated annealing* (dimod/neal) y, opcionalmente, con QAOA en simulador. Se compara honestamente contra el baseline clásico.

Arquitectura híbrida

Los datos de las misiones se ingieren con `rasterio`/`geopandas`, se convierten en un mapa de costos por celda, se resuelve un camino mínimo sobre el grid con A*, se construye una matriz de costos efectivos entre puntos de interés (POIs) y sobre ella se ejecutan en paralelo el solver clásico de OR-Tools y el solver cuántico basado en QUBO. La mejor solución se serializa como GeoJSON, se expone mediante una API REST escrita en FastAPI y se renderiza en un visor web con Leaflet u OpenLayers.

Resultado final esperado

- Un repositorio reproducible con notebooks de cada etapa y scripts ejecutables.
- Una API `GET /ruta` que devuelve un GeoJSON con la traza y la secuencia de POIs.
- Un visor web mínimo que dibuja la ruta sobre el cráter Jezero.
- Un notebook de comparación clásico vs. cuántico con estadísticas (media, desviación, tasa de soluciones factibles, tiempos).
- Una presentación con mensaje claro: *la ventaja cuántica no es hoy práctica a esta escala, pero la formulación es correcta y el pipeline es extensible.*

Principio rector de la planificación

Prioriza siempre **A*** $\succ$ **baseline OR-Tools** $\succ$ **QUBO con SA** $\succ$ **QAOA/D-Wave**. Si el tiempo aprieta en la competencia, recorta por la derecha. Un entregable funcional con las dos primeras capas vale mucho más que un QAOA a medio romper.

2 Arquitectura del sistema

La Figura 1 muestra el flujo completo de datos y procesos. Cada bloque corresponde a un mini entregable independiente y testeable.

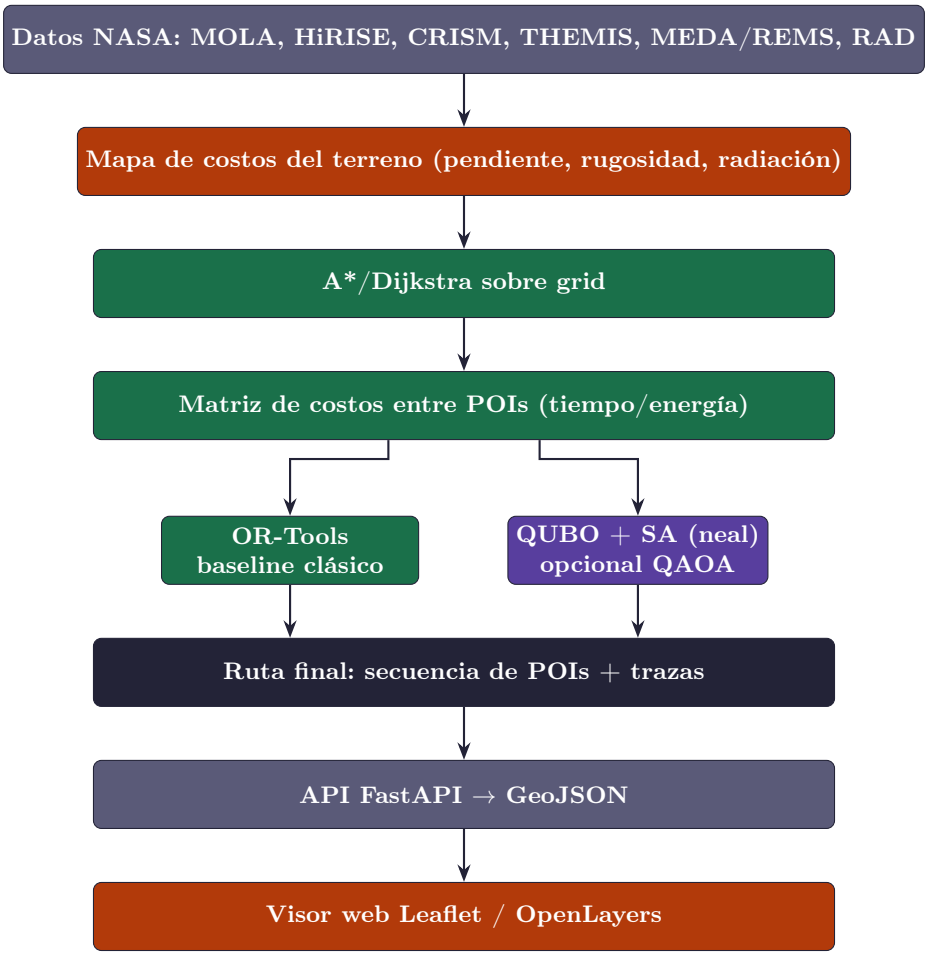


Figura 1: Arquitectura híbrida del algoritmo de ruta (capas clásica, combinatoria y cuántica exploratoria).

Cómo defender la arquitectura ante los jueces

La arquitectura es *modular*: si el módulo cuántico no aporta mejora, el sistema completo sigue funcionando con A* y OR-Tools. Esto no es un defecto, es una decisión de ingeniería que refleja el estado del arte actual.

3 Equipo y contratos de intercambio

El equipo (4 integrantes) se reparte en cuatro roles. Para que nadie bloquee a nadie, se acuerdan desde el inicio los formatos de intercambio.

#	Rol	Responsabilidad principal
1	Algoritmo de ruta (yo)	A*, OR-Tools, QUBO/cuántico, API FastAPI con GeoJSON
2	Datos y geoprocesamiento	Descargar, recortar y reproyectar capas; generar mapa de costos (GeoTIFF) y tiles
3	Frontend y UX	Visor por capas, panel de briefing del astronauta, comparación de rutas
4	Ciencia, producto y narrativa	Sitios de interés y valor científico, límites del traje, validación, video y presentación

De	Hacia	Contrato
Datos (2)	Algoritmo (1)	GeoTIFF del mapa de costos (misma proyección y resolución)
Ciencia (4)	Algoritmo (1)	CSV/JSON con sitios: id, lat, lon, valor científico
Algoritmo (1)	Frontend (3)	GeoJSON con ruta, secuencia de POIs y métricas
Datos (2)	Frontend (3)	Tiles de capas (MOLA, HiRISE, CRISM, THEMIS)

Trabajar en paralelo desde el día 1

Mientras no esté el dato real, yo trabajo con un mapa de costos sintético y el frontend con una ruta GeoJSON de ejemplo. La función de costo es zona compartida con el rol 2: definan juntos los pesos para no pisarse.

4 Roadmap semanal de preparación

El roadmap se organiza en **6 semanas** previas al evento de 48 horas, con una dedicación de 10 horas por semana (2 sesiones de 3 h entre semana y 1 sesión de 4 h el fin de semana). Total estimado: ∼60 horas de preparación.

Verificar la fecha real

Este calendario asume 6 semanas desde el 21 de octubre de 2026. Confirma la fecha oficial del evento en `https://www.spaceappschallenge.org` y desplaza o comprime las semanas según corresponda (si hay menos tiempo, fusiona las semanas 5 y 6 y deja QAOA fuera).

4.1 Semana 1 — 21 al 27 de octubre: datos marcianos y mapa de pendientes

Objetivos de aprendizaje

- Entender qué es un DEM y las diferencias entre MOLA (global, ∼463 m/píxel) y HiRISE (0.25–1 m/píxel en sus modelos de elevación).
- Leer rasters con `rasterio`: CRS, transform, nodata, ventanas.
- Calcular pendiente con `numpy.gradient` usando el espaciado de píxel **en metros**.
- Visualizar mapas con `matplotlib` y exportar GeoJSON con `geopandas`.

Temas y recursos gratuitos recomendados

- Documentación oficial de `rasterio`: https://rasterio.readthedocs.io
- *Mars Trek* de NASA: https://trek.nasa.gov/mars
- Portal HiRISE: https://www.uahirise.org
- Lucas, A. (2014), *Ising formulations of many NP problems*, Frontiers in Physics (lectura temprana de las secciones 1–3).

Mini entregable A — Mapa de pendientes de un DEM de Jezero

Descripción: dado un recorte del DEM de Jezero (tile de MOLA o producto HiRISE), calcular la pendiente en grados y guardar una figura PNG.
Criterios de aceptación:

☐ El DEM se carga con `rasterio.open` y se reportan CRS, resolución y rango de valores.

☐ Si el DEM está en grados, el espaciado de píxel se convierte a **metros** antes de calcular el gradiente (en caso contrario las pendientes salen absurdas).

☐ La pendiente se calcula en grados y se verifica con un caso sintético (plano inclinado o cono con pendiente conocida).

☐ Se genera una figura con barra de color con unidades.

☐ Se exporta el recorte en GeoTIFF al repositorio.

☐ El script corre de extremo a extremo con `python slope_jezero.py`.

Horas estimadas: 8–10 h.

Listing 1: Esqueleto: pendiente en grados con espaciado en metros

```
import numpy as np
import rasterio

with rasterio.open("jezero_dem.tif") as src:
    z = src.read(1).astype("float64")
    dx = abs(src.transform.a)   # tamano de pixel en x (unidades del CRS)
    dy = abs(src.transform.e)   # tamano de pixel en y
    # Si el CRS esta en grados: convertir a metros
    # (Marte: radio medio ~3389.5 km -> 1 grado ~ 59.15 km)

dzdy, dzdx = np.gradient(z, dy, dx)          # dx, dy en METROS
slope_deg = np.degrees(np.arctan(np.hypot(dzdx, dzdy)))
```

Riesgos y plan B

El DEM completo de MOLA puede pesar cientos de MB y no cabe en memoria. **Plan B:** descargar un tile pequeño desde Mars Trek o submuestrear con `rasterio.read(out_shape=...)`. Si HiRISE no está disponible, usar solo MOLA para el prototipo y dejar HiRISE como mejora opcional.

4.2 Semana 2 — 28 de octubre al 3 de noviembre: A* desde cero y validación

Objetivos de aprendizaje

- Comprender A*: costo acumulado $g(n)$, heurística $h(n)$ admisible y consistente, complejidad.
- Implementar una cola de prioridad eficiente con `heapq`.
- Validar contra una implementación de referencia para ganar confianza.

Temas y recursos gratuitos recomendados

- Hart, Nilsson y Raphael (1968), *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*.
- Documentación de `skimage.graph.route_through_array`: https://scikit-image.org/docs/stable/api/skimage.graph.html
- Red Blob Games, introducción interactiva a A*: https://www.redblobgames.com/pathfinding/a-star/introduction.html

Mini entregable B — A* validado contra scikit-image

Descripción: implementar A* desde cero sobre una matriz de costos $M \in \mathbb{R}_{>0}^{H \times W}$, con vecindad 4 u 8, y contrastar el resultado contra `route_through_array`.
Criterios de aceptación:

☐ A* devuelve una ruta y su costo total en una instancia sintética de 100×100.

☐ El costo coincide con el de `route_through_array` con tolerancia 10^{-6} (misma definición de costo de aristas diagonales).

☐ Se prueban al menos 3 instancias (con obstáculos, costos uniformes y aleatoria) y se reportan tiempos.

☐ Hay una prueba unitaria con `pytest`.

Horas estimadas: 8–10 h.

Riesgos y plan B

La heurística euclidiana puede no ser admisible si los costos por celda son < 1. **Plan B:** normalizar la matriz de costos para que el mínimo sea ≥ 1, o escalar la heurística por el costo mínimo de celda.

4.3 Semana 3 — 4 al 10 de noviembre: función de costo y restricciones duras

Objetivos de aprendizaje

- Diseñar una función de costo multicriterio: pendiente, rugosidad (desviación local o TPI), transitabilidad (THEMIS) y radiación (proxy).

- Implementar restricciones duras: pendiente máxima configurable, zonas prohibidas, zonas de baja iluminación.
- Calibrar pesos con un pequeño barrido y documentar la elección.

Temas y recursos gratuitos recomendados

- Artículos de planificación de travesías de MER/MSL (NASA JPL).
- Documentación de THEMIS: `https://themis.asu.edu`
- Mediciones de dosis de RAD (Curiosity) para calibrar el proxy de radiación.

Sobre THEMIS y radiación

THEMIS mide *inercia térmica*: sirve para estimar **transitabilidad** (arena y polvo frente a roca), no radiación. Como proxy de radiación usa la **elevación** (menos atmósfera encima implica más dosis) más el apantallamiento del terreno, calibrado con RAD.

Mini entregable C — Función de costo combinada

Descripción: construir $C(i,j) = w_1\,\mathrm{pend}(i,j) + w_2\,\mathrm{rug}(i,j) + w_3\,\mathrm{trans}(i,j) + w_4\,\mathrm{rad}(i,j)$ con máscaras duras que marquen las celdas inviables.
Criterios de aceptación:

☐ La función acepta un DEM y devuelve una matriz de costos con `np.inf` (o `np.nan` tratado explícitamente) en celdas inviables.

☐ La pendiente máxima es un **parámetro configurable**; el valor por defecto para un astronauta a pie es 15–20° (el umbral de 25° corresponde a rovers).

☐ Se documenta la elección de pesos en un notebook con justificación física.

☐ Se corre A* sobre el mapa de costos y se visualiza la ruta resultante.

Horas estimadas: 10 h.

Riesgos y plan B

La radiación es difícil de modelar sin herramientas de iluminación. **Plan B:** usar solo el proxy de elevación y declararlo como aproximación en el README. En Jezero (∼18° N) no hay sombras permanentes: usa "zonas de baja iluminación" o simplemente omite este factor.

4.4 Semana 4 — 11 al 17 de noviembre: orienteering con OR-Tools y fundamentos cuánticos

Objetivos de aprendizaje

- Entender TSP y el *Orienteering Problem* (OP): maximizar valor científico bajo un presupuesto de oxígeno/tiempo. Idea de NP-dificultad.
- Modelar con OR-Tools (Routing con nodos opcionales y penalización por omitirlos, o CP-SAT).
- Construir la matriz de costos $N \times N$ entre POIs usando A* (Semana 2–3).
- Fundamentos mínimos de cómputo cuántico: qubit, superposición, medición, puertas H/X/CNOT/RX/RY/RZ, annealing vs. compuertas, era NISQ.

Temas y recursos gratuitos recomendados

- Documentación de Google OR-Tools (sección Routing y CP-SAT): https://developers.google.com/optimization
- Qiskit Textbook / IBM Quantum Learning (capítulos de fundamentos).
- Quantum Katas de Microsoft (ejercicios iniciales).

Mini entregable D — Orienteering de 6–8 puntos con OR-Tools (baseline)

Descripción: dado un depósito (punto de salida/regreso), 6–8 POIs con valor científico v_i y una matriz de costos d_{ij}, devolver la secuencia que maximiza el valor total sin exceder el presupuesto.
Criterios de aceptación:

☐ La matriz d_{ij} se calcula con tu A* sobre el mapa de costos real (o sintético).

☐ El solver respeta el presupuesto y devuelve ruta, valor total y costo total.

☐ Se verifica por **fuerza bruta** en instancias de ≤ 8 POIs que el óptimo coincide.

☐ La instancia (CSV/JSON) y la solución quedan versionadas: serán el *benchmark* del módulo cuántico.

☐ Se mide el tiempo de cómputo.

Horas estimadas: 10 h (7 h OR-Tools, 3 h fundamentos cuánticos).

Riesgos y plan B

Si OR-Tools se complica con nodos opcionales, **plan B:** resolver el OP con una formulación ILP en CP-SAT o con una heurística greedy + 2-opt, y usar la fuerza bruta como referencia. El objetivo es tener un baseline correcto, no elegante.

4.5 Semana 5 — 18 al 24 de noviembre: QUBO, simulated annealing y calibración

Objetivos de aprendizaje

- Formulación QUBO y su relación con el modelo de Ising.
- Convertir restricciones en penalizaciones y entender el equilibrio entre pesos.
- Resolver con `dimod` + `neal` (simulated annealing).
- Medir tasa de soluciones válidas y calidad respecto al óptimo clásico.

Temas y recursos gratuitos recomendados

- Lucas (2014), secciones de TSP y problemas de mochila/rutas.
- Documentación de D-Wave Ocean: https://docs.ocean.dwavesys.com
- Ejemplos de TSP y knapsack en el repositorio de D-Wave (GitHub).

Mini entregable E — QUBO progresivo: max-cut, TSP de 4 ciudades, orienteering de 5–6 sitios

Descripción: tres QUBO de dificultad creciente resueltos con `neal.SimulatedAnnealingSampler`.
Criterios de aceptación:

- ☐ Max-cut de 4 nodos: la solución coincide con la de fuerza bruta.
- ☐ TSP de 4 ciudades: se recupera el tour óptimo y se verifica que es una permutación válida.
- ☐ Orienteering de 5–6 sitios (subconjunto de la instancia de la Semana 4): el QUBO se construye desde la matriz d_{ij} y los valores v_i.
- ☐ Existe una función `decode(sample)` que valida factibilidad y reporta qué restricción se viola.

Horas estimadas: 6 h.

Mini entregable F — Calibración de penalizaciones y tasa de factibilidad

Descripción: barrido de los coeficientes de penalización (P_1, P_2, P_3, P_4) y del número de lecturas del annealer.
Criterios de aceptación:

- ☐ Tabla con tasa de soluciones factibles y gap respecto al óptimo de OR-Tools para al menos 5 configuraciones de penalizaciones.
- ☐ Cada configuración se corre ≥ 30 veces con semillas distintas; se reportan media y desviación.
- ☐ Se documenta la regla elegida (p. ej. cada P_k mayor que el máximo beneficio que se obtendría violando la restricción k).
- ☐ Se anota en el README el escalamiento: variables vs. número de sitios.

Horas estimadas: 4 h.

Riesgos y plan B

Penalizaciones mal calibradas producen muchas rutas inválidas. **Plan B:** reducir a 5 sitios, aumentar las penalizaciones y filtrar muestras inválidas en `decode`. Si el QUBO completo no converge, presenta la formulación (Sección 7) y los resultados del TSP de 4 ciudades como prueba de concepto.

4.6 Semana 6 — 25 de noviembre al 1 de diciembre: API, comparativa y QAOA opcional

Objetivos de aprendizaje

- Exponer el pipeline como API con FastAPI y devolver GeoJSON válido.
- Conversión píxel ↔ coordenadas geográficas (transform de `rasterio`).
- Diseñar una comparativa justa (mismas instancias, varias corridas, estadísticas).
- *Opcional:* QAOA en simulador con Qiskit o PennyLane.

Temas y recursos gratuitos recomendados

- Documentación de FastAPI: `https://fastapi.tiangolo.com`
- Especificación GeoJSON (RFC 7946).
- Qiskit Optimization / PennyLane (tutoriales de QAOA).

Mini entregable H — API FastAPI que devuelve GeoJSON

Descripción: endpoint `GET /ruta` que recibe inicio, lista de POIs y presupuesto, y devuelve un `FeatureCollection`.
Criterios de aceptación:

☐ El GeoJSON contiene una `LineString` (traza detallada), `Points` de POIs en orden y propiedades con distancia, tiempo, oxígeno estimado y valor científico.

☐ Parámetro `solver` ∈ {`ortools`, `qubo`}.

☐ El visor del rol 3 dibuja la ruta sin modificaciones.

☐ CORS habilitado y documentación automática en `/docs`.

☐ Existe una ruta de respaldo precalculada (JSON estático) por si falla el servidor en la demo.

Horas estimadas: 4 h.

Mini entregable I — Notebook de comparación clásico vs. cuántico

Descripción: un notebook reproducible que enfrenta OR-Tools, fuerza bruta y QUBO + SA sobre las mismas instancias.
Criterios de aceptación:

☐ Al menos 3 tamaños de instancia (p. ej. 5, 6 y 8 sitios).

☐ ≥ 30 corridas por método estocástico; media, desviación, tasa de factibilidad, gap al óptimo y tiempo.

☐ Gráficas claras (barras con barras de error y una gráfica de escalamiento de variables QUBO).

☐ Una conclusión honesta de media página: qué ganó cada método y por qué.

Horas estimadas: 4 h.

Mini entregable G (opcional) — QAOA en simulador

Descripción: QAOA con profundidad $p = 1, 2$ sobre una instancia mínima (3–4 sitios, ≤ 16 qubits).
Criterios de aceptación:

☐ El circuito se construye desde el mismo QUBO.

☐ Se reporta la probabilidad de medir el óptimo y de medir una solución factible.

☐ Se compara contra SA en la misma instancia.

Horas estimadas: 3–5 h *solo si vas adelantado.*

Riesgos y plan B

La semana 6 es la más cargada. **Plan B:** prioriza H e I; G pasa a "trabajo futuro" en la presentación. Una API estable y una comparativa honesta pesan más que un QAOA incompleto.

5 Tabla resumen de entregables

Sem.	Tema	Entregable	Horas	Est.
1	DEM y pendiente	A: mapa de pendientes de Jezero	8–10	☐
2	A*	B: A* validado vs. scikit-image	8–10	☐
3	Función de costo	C: costo combinado + restricciones duras	10	☐
4	Orienteering + fundamentos cuánticos	D: baseline OR-Tools (6–8 POIs)	10	☐
5	QUBO y SA	E: max-cut, TSP-4, OP 5–6 sitios	6	☐
5	Calibración	F: penalizaciones y tasa de factibilidad	4	☐
6	API	H: FastAPI + GeoJSON	4	☐
6	Comparativa	I: notebook clásico vs. cuántico	4	☐
6	QAOA (opcional)	G: QAOA en simulador	3–5	☐

6 Plan hora por hora de las 48 horas

Horas	Actividad	Hito / punto de control
0–2	Kickoff: fijar sitio, usuario, alcance y contratos de datos	Contratos escritos; repositorio creado
2–6	Subir al repo los módulos preparados (pendiente, A*, costo); conectar con el mapa real del rol 2	**Check 1:** A* corre sobre el mapa real de Jezero
6–12	Matriz de costos entre POIs reales; baseline OR-Tools con valores científicos del rol 4	**Check 2 (h12):** ruta de baseline lista
12–18	Módulo QUBO + SA sobre la instancia real; calibrar penalizaciones	Soluciones factibles $\geq 80\,\%$
18–24	API FastAPI + GeoJSON; integración con el visor	**Check 3 (h24):** integración de punta a punta funcionando
24–30	Comparativa clásico vs. cuántico; métricas para el panel de briefing	Notebook de comparación ejecutable
30–36	Robustez: casos límite, ruta de respaldo precalculada, QAOA solo si hay margen	**Check 4 (h36):** *feature freeze*

Horas	Actividad	Hito / punto de control
36–42	Pulido de interfaz, README, documentación de incertidumbre y fuentes	Repositorio limpio y reproducible
42–46	Video de 30 s, presentación, ensayo del pitch	Video exportado
46–48	Entrega, verificación de enlaces, respaldo en dos lugares	**Entregado**

Qué recortar si voy atrasado

Recorta siempre por la derecha de la prioridad:

1. Si en la hora 12 no hay baseline ⇒ olvidar el QUBO; entregar A* + OR-Tools bien hechos.
2. Si en la hora 24 no hay integración ⇒ congelar funciones y usar la ruta de respaldo precalculada.
3. QAOA/D-Wave real ⇒ solo "trabajo futuro".
4. Nunca recortar: video, README y ensayo del pitch.

7 Formulación matemática del QUBO del orienteering

Sean N sitios con valor científico $v_i > 0$, un depósito 0 (salida y regreso), costos de traslado d_{ij} (calculados con A*), y posiciones $t = 1, \dots, T$ en la secuencia ($T \le N$). Variables binarias:

$$x_{i,t} = \begin{cases} 1 & \text{si el sitio } i \text{ se visita en la posición } t, \\ 0 & \text{en otro caso.} \end{cases}$$

Sea $y_i = \sum_{t=1}^{T} x_{i,t}$ (1 si el sitio i es visitado).

Función objetivo

$$\begin{aligned} H(x) = {} & \underbrace{A\Big[\sum_{t=1}^{T-1}\sum_{i\neq j} d_{ij}\, x_{i,t}\, x_{j,t+1} + \sum_i d_{0i}\, x_{i,1} + \sum_i d_{i0}\, x_{i,T}\Big]}_{\text{costo de traslado}} - \underbrace{B\sum_i v_i\, y_i}_{\text{valor científico}} \\ & + \underbrace{P_1 \sum_i \sum_{t<t'} x_{i,t}\, x_{i,t'}}_{\text{cada sitio a lo sumo una vez}} + \underbrace{P_2 \sum_t \sum_{i<j} x_{i,t}\, x_{j,t}}_{\text{a lo sumo un sitio por posición}} \\ & + \underbrace{P_4 \sum_{t=1}^{T-1} \Big(\sum_j x_{j,t+1}\,(1 - \textstyle\sum_i x_{i,t})\Big)}_{\text{sin huecos en la secuencia}} + \underbrace{P_3 \Big(\sum_i c_i\, y_i + \sum_{k=0}^{K-1} 2^k s_k - R \Big)^2}_{\text{presupuesto con variables de holgura}} \end{aligned} \tag{1}$$

donde R es el presupuesto (oxígeno/tiempo), c_i el costo *lineal* asociado a visitar el sitio i y $s_k \in \{0, 1\}$ son variables de holgura que convierten la desigualdad $\sum_i c_i y_i \le R$ en una igualdad.

Nota importante sobre el presupuesto

El costo exacto de la ruta es *cuadrático* en x; elevarlo al cuadrado en la penalización generaría términos de grado 4, que ya no son QUBO. Por eso el presupuesto se aproxima con costos lineales c_i (p. ej. el costo medio de ir y volver del sitio i más el tiempo de trabajo allí). Tras decodificar, la ruta final se **verifica** contra el presupuesto exacto y se descarta si lo excede. Esta aproximación debe declararse explícitamente.

Reglas para elegir las penalizaciones

- A, B fijan la escala de la función objetivo (normalizar d_{ij} y v_i a $[0, 1]$).
- Cada P_k debe superar el máximo beneficio que se podría ganar violando la restricción k; p. ej. $P_1, P_2 > B \max_i v_i + A \max_{ij} d_{ij}$.
- Penalizaciones demasiado grandes aplanan el paisaje energético y degradan el annealer: se calibran mediante barrido (Mini entregable F).

Tamaño del problema

El número de variables es $N \cdot T + K$ (con $K \approx \lceil \log_2 R \rceil$). Para $N = T = 6$ y $K = 5$ son 41 variables; para $N = T = 8$, 69. Esto es manejable por simulated annealing; QAOA en simulador queda limitado a instancias de ∼15–20 qubits, por eso se usa solo una instancia reducida.

Listing 2: Esqueleto de construccion del QUBO con dimod

```python
import dimod

def build_qubo(d, v, c, R, A, B, P1, P2, P3, P4, T):
    N = len(v)
    bqm = dimod.BinaryQuadraticModel("BINARY")
    x = lambda i, t: f"x_{i}_{t}"
    # 1) valor cientifico (linear, signo negativo)
    for i in range(N):
        for t in range(T):
            bqm.add_variable(x(i, t), -B * v[i])
    # 2) costo de traslado entre posiciones consecutivas
    for t in range(T - 1):
        for i in range(N):
            for j in range(N):
                if i != j:
                    bqm.add_interaction(x(i, t), x(j, t + 1), A * d[i][j])
    # 3) penalizaciones P1, P2, P4 y presupuesto P3 (ver ecuacion)
    # ...
    return bqm
```

8 Limitaciones y honestidad científica

- **No hay ventaja cuántica práctica a esta escala.** Para ≤ 8 sitios un solver clásico encuentra el óptimo en milisegundos. No lo ocultamos: lo medimos y lo reportamos.
- **El camino mínimo sobre una cuadrícula** (A*) no se beneficia de algoritmos cuánticos conocidos; por eso no se usa en esa capa.
- **Hardware actual (era NISQ):** ruido y pocos qubits limitan QAOA a instancias diminutas; el annealing cuántico requiere *minor embedding* que consume muchos qubits físicos.
- **Aproximaciones del modelo:** presupuesto lineal en el QUBO, proxy de radiación, DEM de resolución finita. Todas se declaran y se verifica la solución final con el modelo exacto.

- **Datos:** la resolución y la cobertura varían entre instrumentos; se documenta la incertidumbre por capa.
- **Alcance:** es una herramienta de apoyo a la planificación, no un sistema validado para operaciones reales.

Cómo plantearlo ante los jueces

El problema de seleccionar y ordenar sitios es combinatorio y se mapea de forma natural a QUBO. Implementamos el baseline clásico y la formulación cuántica, los comparamos con rigor, y mostramos cómo escalaría la formulación a decenas de objetivos, donde el hardware cuántico futuro podría ser relevante.

9 Preguntas frecuentes de los jueces

1. **¿Por qué usar cómputo cuántico si el clásico es mejor aquí?**
   Porque el problema se mapea naturalmente a QUBO y queremos evaluar su comportamiento al crecer el número de sitios y restricciones, como en una misión real con muchos objetivos. Reportamos que hoy el clásico gana.

2. **¿Qué pasa si el módulo cuántico falla?**
   El sistema es modular: A* y OR-Tools producen la ruta final de forma independiente.

3. **¿Cómo definieron el valor científico de cada sitio?**
   Con el rol de ciencia, a partir de mineralogía (CRISM), contexto geológico y literatura de la misión Perseverance; es un parámetro editable.

4. **¿Cómo validaron las rutas?**
   Contrastando con trayectos reales de Perseverance/Curiosity y verificando la solución cuántica contra el óptimo por fuerza bruta en instancias pequeñas.

5. **¿Qué incertidumbre tienen los datos?**
   Cada capa documenta resolución, fecha y cobertura; la interfaz marca zonas de baja confianza.

6. **¿Por qué ese umbral de pendiente?**
   Es configurable; usamos un valor conservador para un astronauta a pie con traje y lo diferenciamos del de un rover.

7. **¿Qué escalaría en una misión real?**
   Más POIs, ventanas horarias, clima (MEDA/REMS), dosis de radiación y reoptimización en tiempo real.

10 Checklist final de entrega

☐ Código del repositorio ejecutable desde cero siguiendo el README.

☐ README con arquitectura, instalación, uso, aproximaciones y fuentes de datos.

☐ API funcionando y ruta de respaldo precalculada.

☐ Visor mostrando ruta, POIs y capas de al menos 3–4 misiones.

☐ Notebook de comparación clásico vs. cuántico con estadísticas.

☐ Sección de incertidumbre y limitaciones visible.

☐ Video de 30 s (gancho + demo) exportado y probado.

☐ Presentación final con historia clara (problema, solución, resultado).

☐ Créditos y licencias de datos NASA/USGS correctamente citados.

☐ Entrega subida y verificada; copia de respaldo en dos lugares.

A Apéndice: estructura del repositorio e instalación

Estructura de carpetas sugerida

```
marswalk/
|-- README.md
|-- requirements.txt
|-- data/
|   |-- raw/            # DEM y capas descargadas (no versionar archivos grandes
    )
|   |-- processed/      # recortes GeoTIFF, mapa de costos
|   `-- pois.csv        # sitios: id, lat, lon, valor, costo_visita
|-- src/
|   |-- terrain.py      # pendiente, rugosidad, costo
|   |-- astar.py        # A* desde cero
|   |-- cost_matrix.py  # matriz de costos entre POIs
|   |-- solver_ortools.py
|   |-- solver_qubo.py  # QUBO + simulated annealing
|   |-- solver_qaoa.py  # opcional
|   `-- api.py          # FastAPI
|-- notebooks/
|   |-- 01_slope.ipynb
|   |-- 02_astar_validation.ipynb
|   `-- 09_comparison.ipynb
|-- tests/
|   `-- test_astar.py
`-- web/                # visor (rol 3)
```

Comandos de instalación

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

pip install numpy scipy matplotlib pandas
pip install rasterio geopandas scikit-image networkx
pip install ortools
pip install dimod dwave-neal
pip install fastapi "uvicorn[standard]"
pip install pytest jupyter

# Opcionales (cuantico por compuertas / hardware real)
pip install qiskit qiskit-aer qiskit-optimization
pip install pennylane
pip install dwave-ocean-sdk

# Ejecutar la API
uvicorn src.api:app --reload
```

Nota sobre rasterio en Windows

Si `pip install rasterio` falla, usa `conda` (conda-forge) o trabaja desde Google Colab, donde GDAL ya está disponible.